

# CATO: Charted Axial Transformer Operator

Mathematical Formulation and PyTorch Reference Implementation

## 1 Overview

The Charted Axial Transformer Operator (CATO) is a neural operator designed for learning solution operators of partial differential equations (PDEs) defined on general geometries. Its central idea is to avoid applying attention directly in the physical coordinate system. Instead, CATO learns a continuous latent coordinate chart and performs efficient axial attention conditioned on this learned geometry.

Consider a family of PDEs represented abstractly by

$$\mathcal{L}_a(u) = f, \quad (1)$$

where  $a$  denotes problem-dependent coefficients or input fields,  $f$  denotes forcing terms, and  $u$  is the solution field. The objective of operator learning is to approximate

$$\mathcal{G} : a(\cdot) \mapsto u(\cdot). \quad (2)$$

For a discretized two-dimensional domain, let

$$X = \{x_{ij}\}_{i=1,j=1}^{H,W}, \quad x_{ij} \in \mathbb{R}^2. \quad (3)$$

CATO learns a representation in which the physical geometry is transformed into a latent chart

$$\zeta_{ij} = \Phi_{\text{chart}}(x_{ij}), \quad (4)$$

where

$$\zeta_{ij} = \begin{bmatrix} \xi_{ij} \\ \eta_{ij} \end{bmatrix} \in \mathbb{R}^2. \quad (5)$$

The coordinates  $(\xi, \eta)$  define the learned chart in which axial attention is performed.

## 2 Problem Formulation

Suppose that the physical domain is discretized by an  $H \times W$  mesh. At every mesh point we have physical coordinates

$$x_{ij} = \begin{bmatrix} x_{ij}^{(1)} \\ x_{ij}^{(2)} \end{bmatrix}. \quad (6)$$

Optionally, the model may receive additional local features

$$f_{ij} \in \mathbb{R}^{d_f}. \quad (7)$$

The complete input token is

$$z_{ij}^{\text{in}} = [x_{ij}; f_{ij}] \in \mathbb{R}^{2+d_f}. \quad (8)$$

CATO learns a mapping

$$\mathcal{G}_\theta : (X, F) \longrightarrow \hat{u}, \quad (9)$$

where

$$\hat{u} = \{\hat{u}_{ij}\}_{i,j} \in \mathbb{R}^{H \times W}. \quad (10)$$

In addition to the scalar solution, CATO predicts an auxiliary vector field

$$\hat{q}_{ij} \in \mathbb{R}^2. \quad (11)$$

This auxiliary field acts as a gradient/flux-like representation and allows the training objective to contain derivative-level supervision.

### 3 Input Lifting

The input tokens are lifted into a latent feature space of dimension  $C$ . A two-layer multilayer perceptron performs

$$h_{ij}^{(0)} = \Phi_{\text{pre}}(z_{ij}^{\text{in}}), \quad (12)$$

where

$$\Phi_{\text{pre}} = W_2 \sigma(W_1 z + b_1) + b_2. \quad (13)$$

Therefore,

$$h_{ij}^{(0)} \in \mathbb{R}^C. \quad (14)$$

The latent tensor can be represented as

$$H^{(0)} \in \mathbb{R}^{H \times W \times C}. \quad (15)$$

### 4 Learned Geometry Chart

The defining geometric component of CATO is a learned continuous chart.

Let

$$\Phi_{\text{chart}} : \mathbb{R}^2 \rightarrow \mathbb{R}^2 \quad (16)$$

be a neural mapping from physical coordinates to chart coordinates.

Then

$$\boxed{\zeta_{ij} = \Phi_{\text{chart}}(x_{ij})} \quad (17)$$

with

$$\zeta_{ij} = (\xi_{ij}, \eta_{ij}). \quad (18)$$

A simple parameterization is

$$\Phi_{\text{chart}}(x) = W_2^c \sigma(W_1^c x + b_1^c) + b_2^c. \quad (19)$$

The important conceptual distinction is that  $(x, y)$  are physical coordinates whereas  $(\xi, \eta)$  are learned coordinates optimized for representing the operator.

## 5 Chart-Conditioned Axial Attention

Full self-attention over  $N = HW$  mesh points has quadratic complexity

$$\mathcal{O}(N^2). \quad (20)$$

For a two-dimensional grid this corresponds to

$$\mathcal{O}(H^2W^2). \quad (21)$$

CATO replaces this with axial attention. Attention is applied independently along rows and columns.

Let

$$H \in \mathbb{R}^{H \times W \times C}. \quad (22)$$

For attention head  $m$ , define

$$q_{ij}^{(m)} = W_Q^{(m)} h_{ij}, \quad (23)$$

$$k_{ij}^{(m)} = W_K^{(m)} h_{ij}, \quad (24)$$

$$v_{ij}^{(m)} = W_V^{(m)} h_{ij}. \quad (25)$$

The head dimension is

$$d_h = \frac{C}{M}, \quad (26)$$

where  $M$  is the number of attention heads.

### 5.1 Row Attention

For a fixed row  $i$ , attention is computed across the horizontal coordinate  $j$ .

The chart coordinate associated with the horizontal direction is  $\xi_{ij}$ .

Using a chart-dependent positional transformation  $R(\xi)$ , define

$$\tilde{q}_{ij}^{(m)} = R(\xi_{ij}) q_{ij}^{(m)}, \quad (27)$$

$$\tilde{k}_{ij}^{(m)} = R(\xi_{ij}) k_{ij}^{(m)}. \quad (28)$$

The attention score between positions  $j$  and  $t$  is

$$s_{ijt}^{(m)} = \frac{\left(\tilde{q}_{ij}^{(m)}\right)^T \tilde{k}_{it}^{(m)}}{\sqrt{d_h}}. \quad (29)$$

The normalized attention weight is

$$\alpha_{ijt}^{(m)} = \frac{\exp\left(s_{ijt}^{(m)}\right)}{\sum_{r=1}^W \exp\left(s_{ijr}^{(m)}\right)}. \quad (30)$$

Therefore,

$$A_{\text{row}}^{(m)}(H)_{ij} = \sum_{t=1}^W \alpha_{ijt}^{(m)} v_{it}^{(m)}. \quad (31)$$

After combining the attention heads,

$$A_{\text{row}}(H)_{ij} = W_{\text{row}}^O \text{Concat}_{m=1}^M \left( A_{\text{row}}^{(m)}(H)_{ij} \right). \quad (32)$$

## 5.2 Column Attention

Column attention is defined analogously.

For a fixed column  $j$ , attention is computed across the vertical coordinate  $i$ .

The corresponding chart coordinate is  $\eta_{ij}$ .

The attention score is

$$r_{ijs}^{(m)} = \frac{\left( R(\eta_{ij}) q_{ij}^{(m)} \right)^T \left( R(\eta_{sj}) k_{sj}^{(m)} \right)}{\sqrt{d_h}}. \quad (33)$$

The normalized coefficient is

$$\beta_{ijs}^{(m)} = \frac{\exp(r_{ijs}^{(m)})}{\sum_{p=1}^H \exp(r_{ijp}^{(m)})}. \quad (34)$$

The column attention operator becomes

$$A_{\text{col}}(H)_{ij} = W_{\text{col}}^O \text{Concat}_{m=1}^M \left[ \sum_{s=1}^H \beta_{ijs}^{(m)} v_{sj}^{(m)} \right]. \quad (35)$$

The complete axial attention operator is

$$\boxed{A(H, \zeta) = A_{\text{row}}(H, \xi) + A_{\text{col}}(H, \eta)}. \quad (36)$$

Its computational complexity is approximately

$$\mathcal{O}(H^2W + HW^2), \quad (37)$$

rather than

$$\mathcal{O}(H^2W^2) \quad (38)$$

for unrestricted two-dimensional attention.

## 6 Local Depthwise Operator

Long-range attention is complemented by a local spatial operator.

Let

$$\text{DWConv} \quad (39)$$

denote a depthwise convolution acting independently on feature channels.

The local operator can be written as

$$L(H) = \text{PWConv}(\text{GELU}(\text{DWConv}(H))), \quad (40)$$

where PWConv is a pointwise convolution.

This provides a learned local stencil while the axial attention captures long-range interactions.

## 7 CATO Block

Let  $H^{(\ell)}$  denote the input to block  $\ell$ .

First apply layer normalization:

$$Z^{(\ell)} = \text{LN} \left( H^{(\ell)} \right). \quad (41)$$

The attention and local operators produce

$$\tilde{H}^{(\ell)} = H^{(\ell)} + A \left( Z^{(\ell)}, \zeta \right) + L \left( Z^{(\ell)} \right). \quad (42)$$

The feed-forward transformation then gives

$$H^{(\ell+1)} = \tilde{H}^{(\ell)} + \text{MLP} \left( \text{LN} \left( \tilde{H}^{(\ell)} \right) \right). \quad (43)$$

For  $L$  stacked blocks,

$$H^{(L)} = \text{CATO}_L \circ \dots \circ \text{CATO}_1 \left( H^{(0)}, \zeta \right). \quad (44)$$

## 8 Output Heads

The scalar solution is obtained through a linear projection

$$\boxed{\hat{u}_{ij} = W_u h_{ij}^{(L)} + b_u}. \quad (45)$$

The auxiliary vector field is

$$\boxed{\hat{q}_{ij} = W_q h_{ij}^{(L)} + b_q}. \quad (46)$$

For a two-dimensional domain,

$$\hat{q}_{ij} = \begin{bmatrix} \hat{q}_{ij}^{(1)} \\ \hat{q}_{ij}^{(2)} \end{bmatrix}. \quad (47)$$

## 9 Derivative-Aware Physical Supervision

A key feature of CATO is that training is not restricted to pointwise solution values.

Suppose that the target solution is  $u$ .

The value loss can be represented using the relative  $L_2$  error:

$$\mathcal{L}_{\text{val}} = \frac{\|\hat{u} - u\|_2}{\|u\|_2 + \epsilon}. \quad (48)$$

### 9.1 Discrete Physical Gradients

For a structured mesh, central differences provide a discrete approximation of spatial derivatives. For example,

$$\Delta_i u_{ij} = u_{i+1,j} - u_{i-1,j}, \quad (49)$$

and

$$\Delta_j u_{ij} = u_{i,j+1} - u_{i,j-1}. \quad (50)$$

If the physical coordinates are nonuniform, the coordinate differences must also be taken into account. Conceptually,

$$u_x \approx \frac{\Delta_i u}{\Delta_i x}, \quad u_y \approx \frac{\Delta_j u}{\Delta_j y}. \quad (51)$$

Thus,

$$\nabla_h u = \begin{bmatrix} D_x u \\ D_y u \end{bmatrix}. \quad (52)$$

The gradient loss is

$$\mathcal{L}_{\text{grad}} = \|\nabla_h \hat{u} - \nabla_h u\|_2^2. \quad (53)$$

### 9.2 Auxiliary Flux Loss

The auxiliary prediction  $\hat{q}$  is directly supervised against the target gradient or corresponding flux-like quantity:

$$\mathcal{L}_{\text{flux}} = \|\hat{q} - \nabla_h u\|_2^2. \quad (54)$$

### 9.3 Consistency Loss

The auxiliary field should also remain consistent with the gradient of the predicted solution:

$$\mathcal{L}_{\text{cons}} = \|\hat{q} - \nabla_h \hat{u}\|_2^2. \quad (55)$$

### 9.4 Total Objective

The complete training objective is

$$\boxed{\mathcal{L} = \mathcal{L}_{\text{val}} + \lambda_g \mathcal{L}_{\text{grad}} + \lambda_f \mathcal{L}_{\text{flux}} + \lambda_c \mathcal{L}_{\text{cons}}}. \quad (56)$$

Here,

$$\lambda_g \geq 0, \quad (57)$$

$$\lambda_f \geq 0, \quad (58)$$

$$\lambda_c \geq 0 \quad (59)$$

control the contribution of derivative and consistency supervision.

## 10 Charted Low-Rank Operator Interpretation

The learned chart is not merely a positional encoding. Its purpose is to make the underlying operator easier to represent.

Consider an operator  $\tilde{\mathcal{G}}$  that admits the decomposition

$$\tilde{\mathcal{G}} = T_{\zeta} + R, \quad (60)$$

where  $T_{\zeta}$  is approximately separable along the learned chart coordinates and  $R$  is a residual operator satisfying

$$\|Rf\|_2 \leq \varepsilon_{\text{rk}} \|f\|_2. \quad (61)$$

A charted axial low-rank representation can be written schematically as

$$\begin{aligned} (T_{\zeta}f)_{ij} = & \sum_{r=1}^{R_{\xi}} a_r(\zeta_{ij}) \left[ \frac{1}{W} \sum_{t=1}^W b_r(\zeta_{it}) f_{it} \right] \\ & + \sum_{s=1}^{R_{\eta}} c_s(\zeta_{ij}) \left[ \frac{1}{H} \sum_{p=1}^H d_s(\zeta_{pj}) f_{pj} \right] + \ell(\zeta_{ij}) f_{ij}. \end{aligned} \quad (62)$$

The first summation describes interactions along the  $\xi$  direction, while the second describes interactions along  $\eta$ .

If the learned chart is perturbed,

$$\max_{i,j} \left\| \hat{\zeta}_{ij} - \zeta_{ij} \right\| \leq \delta, \quad (63)$$

then under suitable Lipschitz assumptions the approximation degradation can be bounded by

$$\left\| T_{\hat{\zeta}}f - T_{\zeta}f \right\|_2 \leq C_{\text{chart}} \delta \|f\|_2. \quad (64)$$

Consequently, for  $\|f\|_2 \leq M$ , a representative approximation bound is

$$\sup_{\|f\|_2 \leq M} \left\| N_{\Theta}(f, X) - \tilde{\mathcal{G}}f \right\|_2 \leq \varepsilon_{\text{rk}} M + C_{\text{chart}} M \delta + \varepsilon_{\text{nn}}. \quad (65)$$

The three contributions have the interpretation

$$\underbrace{\varepsilon_{\text{rk}} M}_{\text{operator low-rank approximation}} + \underbrace{C_{\text{chart}} M \delta}_{\text{chart error}} + \underbrace{\varepsilon_{\text{nn}}}_{\text{neural approximation error}}. \quad (66)$$

This gives a mathematical motivation for learning the chart: if the physical operator becomes approximately separable in the learned coordinates, the effective complexity of the operator can be reduced.

## 11 CATO as a Neural Operator

The complete architecture can therefore be summarized as

$$(X, F) \xrightarrow{\Phi_{\text{pre}}} H^{(0)} \quad (67)$$

and simultaneously

$$X \xrightarrow{\Phi_{\text{chart}}} \zeta. \quad (68)$$

The latent representation is subsequently transformed through  $L$  CATO blocks:

$$H^{(L)} = \text{CATO}_{\theta} \left( H^{(0)}, \zeta \right). \quad (69)$$

Finally,

$$H^{(L)} \xrightarrow{W_u} \hat{u}, \quad (70)$$

and

$$H^{(L)} \xrightarrow{W_q} \hat{q}. \quad (71)$$

Thus the complete learned operator is

$$\boxed{\mathcal{G}_{\theta}(X, F) = (\hat{u}, \hat{q})}. \quad (72)$$

## 12 PyTorch Reference Implementation

The following implementation realizes the principal CATO components: learned coordinate charts, chart-conditioned axial attention, local depthwise operators, residual CATO blocks, scalar solution prediction, and auxiliary vector-field prediction.

This is a reference implementation of the mathematical architecture described above and is not intended to reproduce every implementation detail of the authors' experimental code.

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5
6 # =====
7 # MLP
8 # =====
9
10 class MLP(nn.Module):
11     def __init__(self, dim, hidden_dim, out_dim=None):
12         super().__init__()
13
14         if out_dim is None:
15             out_dim = dim
16
17         self.net = nn.Sequential(
18             nn.Linear(dim, hidden_dim),
19             nn.GELU(),
20             nn.Linear(hidden_dim, out_dim)
21         )
22
23     def forward(self, x):
24         return self.net(x)
25
26
27 # =====
28 # Learned Continuous Chart
29 # =====
30

```

```

31 class ChartNetwork(nn.Module):
32     """
33     Maps physical coordinates (x, y) to learned chart
34     coordinates (xi, eta).
35     """
36
37     def __init__(self, hidden_dim=64):
38         super().__init__()
39
40         self.net = nn.Sequential(
41             nn.Linear(2, hidden_dim),
42             nn.GELU(),
43             nn.Linear(hidden_dim, hidden_dim),
44             nn.GELU(),
45             nn.Linear(hidden_dim, 2)
46         )
47
48     def forward(self, coords):
49         # coords: [B, H, W, 2]
50         return self.net(coords)
51
52
53 # =====
54 # Chart-conditioned axial attention
55 # =====
56
57 class AxialAttention(nn.Module):
58
59     def __init__(self, dim, heads=4):
60         super().__init__()
61
62         assert dim % heads == 0
63
64         self.dim = dim
65         self.heads = heads
66         self.head_dim = dim // heads
67
68         self.qkv_row = nn.Linear(dim, 3 * dim)
69         self.qkv_col = nn.Linear(dim, 3 * dim)
70
71         self.row_out = nn.Linear(dim, dim)
72         self.col_out = nn.Linear(dim, dim)
73
74     def split_heads(self, x):
75         # [B, L, C] -> [B, heads, L, head_dim]
76         B, L, C = x.shape
77
78         x = x.view(
79             B,
80             L,
81             self.heads,
82             self.head_dim
83         )
84
85         return x.transpose(1, 2)
86
87     def chart_bias(self, coord):
88         """

```

```

89     Simple relative chart-distance bias.
90
91     coord:
92         [B, L]
93
94     returns:
95         [B, 1, L, L]
96     """
97
98     distance = (
99         coord.unsqueeze(-1)
100         -
101         coord.unsqueeze(-2)
102     )
103
104     bias = -distance.abs()
105
106     return bias.unsqueeze(1)
107
108 def row_attention(self, x, xi):
109
110     B, H, W, C = x.shape
111
112     # Process each row independently.
113     x_row = x.reshape(B * H, W, C)
114
115     qkv = self.qkv_row(x_row)
116
117     q, k, v = torch.chunk(
118         qkv,
119         3,
120         dim=-1
121     )
122
123     q = self.split_heads(q)
124     k = self.split_heads(k)
125     v = self.split_heads(v)
126
127     scores = torch.matmul(
128         q,
129         k.transpose(-2, -1)
130     )
131
132     scores = scores / (self.head_dim ** 0.5)
133
134     xi_row = xi.reshape(B * H, W)
135
136     bias = self.chart_bias(xi_row)
137
138     scores = scores + bias
139
140     attn = torch.softmax(
141         scores,
142         dim=-1
143     )
144
145     out = torch.matmul(
146         attn,

```

```

147         v
148     )
149
150     out = out.transpose(1, 2)
151
152     out = out.reshape(
153         B * H,
154         W,
155         C
156     )
157
158     out = self.row_out(out)
159
160     return out.reshape(B, H, W, C)
161
162 def col_attention(self, x, eta):
163
164     B, H, W, C = x.shape
165
166     # Transpose so each column becomes a sequence.
167     x_col = (
168         x.permute(0, 2, 1, 3)
169         .contiguous()
170         .reshape(B * W, H, C)
171     )
172
173     qkv = self.qkv_col(x_col)
174
175     q, k, v = torch.chunk(
176         qkv,
177         3,
178         dim=-1
179     )
180
181     q = self.split_heads(q)
182     k = self.split_heads(k)
183     v = self.split_heads(v)
184
185     scores = torch.matmul(
186         q,
187         k.transpose(-2, -1)
188     )
189
190     scores = scores / (self.head_dim ** 0.5)
191
192     eta_col = (
193         eta.permute(0, 2, 1)
194         .contiguous()
195         .reshape(B * W, H)
196     )
197
198     bias = self.chart_bias(eta_col)
199
200     scores = scores + bias
201
202     attn = torch.softmax(
203         scores,
204         dim=-1

```

```

205         )
206
207         out = torch.matmul(
208             attn,
209             v
210         )
211
212         out = out.transpose(1, 2)
213
214         out = out.reshape(
215             B * W,
216             H,
217             C
218         )
219
220         out = self.col_out(out)
221
222         out = out.reshape(
223             B,
224             W,
225             H,
226             C
227         )
228
229         return out.permute(
230             0, 2, 1, 3
231         )
232
233     def forward(self, x, chart):
234
235         xi = chart[..., 0]
236         eta = chart[..., 1]
237
238         row = self.row_attention(
239             x,
240             xi
241         )
242
243         col = self.col_attention(
244             x,
245             eta
246         )
247
248         return row + col
249
250
251     # =====
252     # Local depthwise operator
253     # =====
254
255     class LocalOperator(nn.Module):
256
257         def __init__(self, dim):
258             super().__init__()
259
260             self.depthwise = nn.Conv2d(
261                 dim,
262                 dim,

```

```

263         kernel_size=3,
264         padding=1,
265         groups=dim
266     )
267
268     self.pointwise = nn.Conv2d(
269         dim,
270         dim,
271         kernel_size=1
272     )
273
274     def forward(self, x):
275
276         # [B, H, W, C] -> [B, C, H, W]
277         x = x.permute(
278             0, 3, 1, 2
279         )
280
281         x = self.depthwise(x)
282
283         x = F.gelu(x)
284
285         x = self.pointwise(x)
286
287         # [B, C, H, W] -> [B, H, W, C]
288         return x.permute(
289             0, 2, 3, 1
290         )
291
292
293     # =====
294     # CATO Block
295     # =====
296
297     class CATOBlock(nn.Module):
298
299         def __init__(
300             self,
301             dim,
302             heads=4,
303             mlp_ratio=4.0
304         ):
305             super().__init__()
306
307             self.norm1 = nn.LayerNorm(dim)
308             self.norm2 = nn.LayerNorm(dim)
309
310             self.attention = AxialAttention(
311                 dim=dim,
312                 heads=heads
313             )
314
315             self.local = LocalOperator(
316                 dim
317             )
318
319             hidden_dim = int(
320                 dim * mlp_ratio

```

```

321         )
322
323         self.mlp = MLP(
324             dim=dim,
325             hidden_dim=hidden_dim
326         )
327
328     def forward(self, x, chart):
329
330         z = self.norm1(x)
331
332         x = (
333             x
334             +
335             self.attention(z, chart)
336             +
337             self.local(z)
338         )
339
340         x = x + self.mlp(
341             self.norm2(x)
342         )
343
344         return x
345
346
347     # =====
348     # CATO
349     # =====
350
351     class CATO(nn.Module):
352
353     def __init__(
354         self,
355         in_features,
356         hidden_dim=64,
357         depth=4,
358         heads=4
359     ):
360         super().__init__()
361
362         # Input lifting
363         self.input_projection = MLP(
364             dim=in_features,
365             hidden_dim=hidden_dim,
366             out_dim=hidden_dim
367         )
368
369         # Learned coordinate chart
370         self.chart = ChartNetwork(
371             hidden_dim=hidden_dim
372         )
373
374         # CATO blocks
375         self.blocks = nn.ModuleList([
376             CATOBlock(
377                 dim=hidden_dim,
378                 heads=heads

```

```

379         )
380         for _ in range(depth)
381     ])
382
383     # Scalar solution head
384     self.solution_head = nn.Linear(
385         hidden_dim,
386         1
387     )
388
389     # Auxiliary vector-field head
390     self.flux_head = nn.Linear(
391         hidden_dim,
392         2
393     )
394
395     def forward(
396         self,
397         coords,
398         features
399     ):
400
401         # -----
402         # Input
403         # -----
404
405         # coords:
406         # [B, H, W, 2]
407
408         # features:
409         # [B, H, W, d_f]
410
411         x = torch.cat(
412             [
413                 coords,
414                 features
415             ],
416             dim=-1
417         )
418
419         # -----
420         # Input lifting
421         # -----
422
423         x = self.input_projection(x)
424
425         # -----
426         # Learned geometry
427         # -----
428
429         chart = self.chart(coords)
430
431         # -----
432         # CATO blocks
433         # -----
434
435         for block in self.blocks:
436             x = block(

```

```

437         x,
438         chart
439     )
440
441     # -----
442     # Output fields
443     # -----
444
445     solution = self.solution_head(x)
446
447     flux = self.flux_head(x)
448
449     return {
450         "solution": solution,
451         "flux": flux,
452         "chart": chart
453     }
454
455     # =====
456     # Discrete gradient
457     # =====
458
459
460 def discrete_gradient(u, coords):
461
462     """
463     Computes a simple finite-difference gradient.
464
465     u:
466         [B, H, W, 1]
467
468     coords:
469         [B, H, W, 2]
470     """
471
472     u = u[..., 0]
473
474     dx = coords[:, 1:, :, 0] - \
475         coords[:, :-1, :, 0]
476
477     dy = coords[:, :, 1:, 1] - \
478         coords[:, :, :-1, 1]
479
480     du_dx = (
481         u[:, 1:, :] -
482         u[:, :-1, :]
483     ) / (dx + 1e-8)
484
485     du_dy = (
486         u[:, :, 1:] -
487         u[:, :, :-1]
488     ) / (dy + 1e-8)
489
490     # Put gradients on a common interior grid.
491     du_dx = du_dx[:, :, :-1]
492     du_dy = du_dy[:, :-1, :]
493
494     return torch.stack(

```

```

495         [du_dx, du_dy],
496         dim=-1
497     )
498
499
500 # =====
501 # Relative L2 loss
502 # =====
503
504 def relative_l2(pred, target):
505
506     numerator = torch.norm(
507         pred - target
508     )
509
510     denominator = (
511         torch.norm(target)
512         + 1e-8
513     )
514
515     return numerator / denominator
516
517
518 # =====
519 # Derivative-aware CATO loss
520 # =====
521
522 def cato_loss(
523     prediction,
524     target_solution,
525     coords,
526     lambda_grad=1.0,
527     lambda_flux=1.0,
528     lambda_cons=1.0
529 ):
530
531     predicted_solution = prediction[
532         "solution"
533     ]
534
535     predicted_flux = prediction[
536         "flux"
537     ]
538
539     # -----
540     # Value loss
541     # -----
542
543     loss_value = relative_l2(
544         predicted_solution,
545         target_solution
546     )
547
548     # -----
549     # Gradient supervision
550     # -----
551
552     pred_grad = discrete_gradient(

```

```

553         predicted_solution,
554         coords
555     )
556
557     true_grad = discrete_gradient(
558         target_solution,
559         coords
560     )
561
562     loss_grad = F.mse_loss(
563         pred_grad,
564         true_grad
565     )
566
567     # -----
568     # Flux supervision
569     # -----
570
571     # Match dimensions to interior grid.
572     pred_flux = predicted_flux[
573         :, :-1, :-1, :
574     ]
575
576     loss_flux = F.mse_loss(
577         pred_flux,
578         true_grad
579     )
580
581     # -----
582     # Consistency
583     # -----
584
585     loss_cons = F.mse_loss(
586         pred_flux,
587         pred_grad
588     )
589
590     # -----
591     # Total loss
592     # -----
593
594     total = (
595         loss_value
596         +
597         lambda_grad * loss_grad
598         +
599         lambda_flux * loss_flux
600         +
601         lambda_cons * loss_cons
602     )
603
604     return {
605         "total": total,
606         "value": loss_value,
607         "gradient": loss_grad,
608         "flux": loss_flux,
609         "consistency": loss_cons
610     }

```

```

611
612
613 # =====
614 # Example
615 # =====
616
617 if __name__ == "__main__":
618
619     device = (
620         "cuda"
621         if torch.cuda.is_available()
622         else "cpu"
623     )
624
625     B = 4
626     H = 32
627     W = 32
628     D = 3
629
630     # Physical coordinates
631     x = torch.linspace(
632         0.0,
633         1.0,
634         W
635     )
636
637     y = torch.linspace(
638         0.0,
639         1.0,
640         H
641     )
642
643     yy, xx = torch.meshgrid(
644         y,
645         x,
646         indexing="ij"
647     )
648
649     coords = torch.stack(
650         [xx, yy],
651         dim=-1
652     )
653
654     coords = coords.unsqueeze(0).repeat(
655         B, 1, 1, 1
656     )
657
658     # Example input field
659     features = torch.randn(
660         B,
661         H,
662         W,
663         D
664     )
665
666     # Example target solution
667     target = torch.randn(
668         B,

```

```

669         H,
670         W,
671         1
672     )
673
674     model = CATO(
675         in_features=2 + D,
676         hidden_dim=64,
677         depth=4,
678         heads=4
679     ).to(device)
680
681     coords = coords.to(device)
682     features = features.to(device)
683     target = target.to(device)
684
685     prediction = model(
686         coords,
687         features
688     )
689
690     losses = cato_loss(
691         prediction,
692         target,
693         coords
694     )
695
696     print(
697         "Total loss:",
698         losses["total"].item()
699     )
700
701     print(
702         "Solution shape:",
703         prediction["solution"].shape
704     )
705
706     print(
707         "Flux shape:",
708         prediction["flux"].shape
709     )
710
711     print(
712         "Chart shape:",
713         prediction["chart"].shape
714     )

```

## 13 Interpretation

CATO can therefore be viewed as combining four principal ideas:

1. **Geometry adaptation:** physical coordinates are mapped to a learned latent chart.
2. **Efficient nonlocal interaction:** axial attention captures long-range interactions along the two chart directions without requiring full two-dimensional attention.
3. **Local interaction:** depthwise convolution provides a learned local spatial operator.

4. **Derivative-aware supervision:** the model is trained using solution values, spatial derivatives, an auxiliary vector field, and consistency between the auxiliary field and the predicted gradient.

The resulting architecture is therefore not simply a Transformer applied to a PDE grid. Its defining feature is the attempt to learn a coordinate system in which the operator has a simpler structure.

$$\boxed{\text{Physical geometry} \rightarrow \text{Learned chart} \rightarrow \text{Axial operator learning} \rightarrow \text{Physical field}} \quad (73)$$

## 14 Reference

- C.-W. Cheng, S. Wang, C.-B. Schönlieb, and A. I. Avilés-Rivero, *CATO: Charted Attention for Neural PDE Operators*, arXiv:2605.09016, 2026.